07_metabolomics

September 3, 2026

METHODS WE WILL USE

1)Preprocessing chain (normalization → log → Pareto scaling)

2)PLS regression in the body_score + LOO-CV

3)PLS-DA body_class + LOO-CV (feasibility check)

4)VIP scores

5)Permutation test Q^2 (mandatory with n=17)

6)MLflow logging

data from Skogerson, K., Runnebaum, R., Wohlgemuth, G., De Ropp, J., Heymann, H., & Fiehn, O. (2009). Comparison of Gas Chromatography-Coupled Time-of-Flight Mass Spectrometry and1 H Nuclear Magnetic Resonance Spectroscopy Metabolite Identification in White Wines from a Sensory Study Investigating Wine Body. Journal of Agricultural and Food Chemistry, 57(15), 6899–6907. https://doi.org/10.1021/jf9019322

[1]:
```
import pandas as pd

ann = pd.read_excel("/home/nasia/wine-innovation-engine/data/processed/
 ↪summary_421340.xls", sheet_name="Annotation Data")

ann.head(30)
```

[1]:

	SampleID	ClassID	1,2-anhydro-myo-inositol	1-hexadecanol	\
0	421474	421495	21306	2532.0	
1	421478	421495	9893	3347.0	
2	421482	421495	23393	3552.0	
3	421486	421495	21540	2369.0	
4	421490	421495	20527	2817.0	
5	421494	421495	21072	4239.0	
6	421499	421520	9157	1573.0	
7	421503	421520	12483	1463.0	
8	421507	421520	11861	4842.0	
9	421511	421520	9216	1618.0	
10	421515	421520	11331	1305.0	
11	421519	421520	10779	3065.0	
12	421524	421545	12111	2367.0	

13	421528	421545	10957	3826.0
14	421532	421545	11625	4676.0
15	421536	421545	12059	6183.0
16	421540	421545	12571	5142.0
17	421544	421545	12335	2202.0
18	421549	421570	14887	2478.0
19	421553	421570	12951	3027.0
20	421557	421570	10367	4955.0
21	421561	421570	12253	3406.0
22	421565	421570	10888	3921.0
23	421569	421570	9856	1926.0
24	421574	421595	18681	1655.0
25	421578	421595	14278	3898.0
26	421582	421595	17095	3950.0
27	421586	421595	15634	2120.0
28	421590	421595	13635	3943.0
29	421594	421595	17105	3266.0

	2-hydroxyglutaric acid	2-isopropylmalic acid	2-ketoisocaproic acid	\
0	121483	6570	748.0	
1	125207	6900	439.0	
2	120217	6936	909.0	
3	123227	7455	790.0	
4	117192	6418	517.0	
5	113118	6389	607.0	
6	104713	6663	943.0	
7	95385	6134	549.0	
8	94267	6088	590.0	
9	101264	6360	646.0	
10	94999	6206	827.0	
11	99510	6194	816.0	
12	106165	4946	743.0	
13	96933	4507	682.0	
14	100556	4778	1048.0	
15	100358	4215	1091.0	
16	106231	5035	781.0	
17	106950	5202	734.0	
18	69757	4976	1049.0	
19	69122	5407	737.0	
20	66217	4369	1070.0	
21	75967	5334	975.0	
22	61527	4547	995.0	
23	75410	5177	798.0	
24	40195	2804	720.0	
25	32018	2333	920.0	
26	37423	2513	521.0	
27	27826	1814	803.0	

28	27305	1847	697.0	
29	35395	2660	652.0	

	226091.0	3,6-anhydrogalactose	3-hydroxy-3-methylglutaric acid	…	\
0	1100.0	3388	2487.0	…	
1	973.0	3418	6556.0	…	
2	916.0	3391	5911.0	…	
3	1080.0	3566	5581.0	…	
4	1009.0	3598	3941.0	…	
5	1237.0	2955	3345.0	…	
6	1454.0	1534	4743.0	…	
7	851.0	1571	3980.0	…	
8	1409.0	1406	3976.0	…	
9	1070.0	1706	4385.0	…	
10	1163.0	1384	2790.0	…	
11	1340.0	1778	4411.0	…	
12	1425.0	3872	3684.0	…	
13	1360.0	3479	3063.0	…	
14	1752.0	3902	3085.0	…	
15	1406.0	3617	3533.0	…	
16	1628.0	4210	2907.0	…	
17	1345.0	3820	3332.0	…	
18	2069.0	2744	1717.0	…	
19	1579.0	2578	2788.0	…	
20	1508.0	2637	3327.0	…	
21	1276.0	2935	1739.0	…	
22	1754.0	2330	3688.0	…	
23	1245.0	3115	3237.0	…	
24	1285.0	8373	2473.0	…	
25	1443.0	7097	2238.0	…	
26	1309.0	8314	2435.0	…	
27	2723.0	6287	653.0	…	
28	1061.0	6020	1771.0	…	
29	880.0	7835	2307.0	…	

	trehalose	tryptophan	tyrosine	uracil	urea	uridine	valine	\
0	8672	3630.0	35717	12905.0	12456.0	188.0	40278	
1	8309	3730.0	37511	11990.0	14406.0	155.0	36145	
2	7337	3971.0	36512	12669.0	15254.0	142.0	37561	
3	5817	3159.0	35175	14331.0	12840.0	188.0	36619	
4	8367	4088.0	36575	10781.0	11471.0	245.0	32712	
5	7904	3559.0	31896	8899.0	9768.0	269.0	34731	
6	6587	504.0	21388	12620.0	2719.0	72.0	20312	
7	5021	624.0	19793	2449.0	2440.0	136.0	19788	
8	4842	674.0	20714	8919.0	3436.0	222.0	19190	
9	5104	461.0	19614	514.0	2622.0	140.0	18245	
10	6510	613.0	17754	9544.0	5519.0	88.0	26095	

11	3581	512.0	19601	8257.0	2741.0	187.0	19534
12	6673	2463.0	25608	18331.0	2529.0	272.0	42484
13	12271	2737.0	25959	12854.0	1948.0	298.0	36113
14	9845	2133.0	27159	12026.0	2326.0	223.0	39373
15	11130	2472.0	27050	11421.0	2302.0	171.0	39208
16	12576	2602.0	27330	12342.0	2158.0	203.0	40806
17	8055	2232.0	27478	16931.0	2405.0	222.0	42554
18	4847	1528.0	27032	8477.0	3485.0	165.0	34206
19	4497	1655.0	30997	8522.0	2766.0	202.0	34064
20	4816	1495.0	28785	9222.0	4261.0	171.0	35032
21	5513	1812.0	32896	15665.0	4139.0	136.0	34214
22	3876	1525.0	30022	9967.0	3035.0	244.0	33235
23	2859	1742.0	30429	9570.0	2598.0	157.0	34257
24	3521	3212.0	13049	5420.0	2902.0	151.0	42761
25	592	2318.0	9120	8275.0	2962.0	112.0	43321
26	2621	3157.0	10768	7168.0	3425.0	106.0	42382
27	2149	2194.0	11097	6795.0	4312.0	150.0	37098
28	1899	1892.0	10437	5786.0	2101.0	92.0	31620
29	2566	3042.0	10509	8126.0	3351.0	129.0	38572

	xanthine	xylitol	xylose
0	821.0	5912	27398
1	908.0	5951	26375
2	1102.0	6365	28070
3	701.0	6331	28590
4	949.0	5508	26948
5	833.0	5178	24976
6	746.0	3207	22878
7	491.0	3075	22873
8	547.0	2923	21212
9	597.0	3162	22748
10	701.0	2504	20616
11	560.0	3130	22836
12	953.0	2503	25167
13	838.0	2647	21006
14	840.0	2676	25081
15	873.0	2855	21933
16	909.0	2623	23779
17	892.0	2699	25320
18	509.0	3434	22650
19	545.0	3812	25158
20	623.0	2613	20474
21	701.0	3475	24220
22	844.0	3152	20828
23	787.0	3266	25077
24	793.0	3784	35911
25	654.0	3336	26705

```
26      661.0      3890    31639
27      686.0      3034    19527
28      598.0      2380    27050
29      780.0      3579    30994

[30 rows x 111 columns]
```

[2]:
```python
import pandas as pd
import numpy as np


#X needs to be a pandas dataframe with 17 WINES and 109 metabolites ,we have␣
 ↪GC-TOF-MS peak intensity ,mean of the 6 replicates for each wine and␣
 ↪metabolite,the data ->normalised peak intensity

def check_zeros(X:pd.DataFrame)->None:
    n_zero_total=(X==0).sum().sum() #sum over columns and total
    n_nan_total=X.isna().sum().sum() #sum over columns and total
    print(f"Total number of zeros in the dataset: {n_zero_total}")
    print(f"Total number of NaN values in the dataset: {n_nan_total}")
    n_cells=X.shape[0]*X.shape[1] #total number of cells in the dataset
    print(f"Percentage of zeros in the dataset: {n_zero_total/n_cells*100:.
 ↪2f}%")
    print(f"Percentage of NaN values in the dataset: {n_nan_total/n_cells*100:.
 ↪2f}%")


    zeros_per_col=(X==0).sum() #sum over rows for each column
    cols_with_zeros=zeros_per_col[zeros_per_col>0].sort_values(ascending=False)␣
 ↪#the head will shows the worst metabolites with the most zeros
    print(f"Number of columns with zeros: {len(cols_with_zeros)}")
    print(f"head of the metabolites with the most zeros:\n{cols_with_zeros.
 ↪head()}")

    print(f"columns with at least one zero: {len(cols_with_zeros)}/{X.
 ↪shape[1]}")

    if len(cols_with_zeros)>0:
        print(cols_with_zeros.head(10))


def log_transform(X:pd.DataFrame,offset:float=1)->pd.DataFrame:
    #log transform the data, we will use log10(x+1) to avoid log(0)
       return np.log10(X+offset)

def pareto_scale(X:pd.DataFrame)->pd.DataFrame:
```

```
    #pareto scaling, we will use the standard deviation of each metabolite to␣
 ↪scale the data
    return (X-X.mean())/np.sqrt(X.std())
```

[ ]:

[3]:
```
#classID WITH body score and body class from the paper

xwalk = {421745:('CH01',2.57,'high'),421770:('CH02',2.85,'high'),421795:
 ↪('CH03',3.21,'high'),
 421820:('CH04',2.55,'high'),421845:('CH05',2.91,'high'),421870:('CH06',2.
 ↪46,'medium'),
 421545:('PG01',2.13,'low'),421570:('PG02',2.24,'low'),421595:('R01',2.
 ↪44,'medium'),
 421620:('R02',2.63,'high'),421645:('SB01',2.16,'low'),421670:('SB02',1.
 ↪97,'low'),
 421695:('SB03',2.33,'medium'),421720:('SB04',2.12,'low'),421495:('V01',2.
 ↪42,'medium'),
 421520:('V02',2.32,'medium'),421895:('WW01',2.22,'low')}

#orient ='index' means that the keys of the dictionary will be used as the␣
 ↪index of the DataFrame, and the values will be used as the data for the␣
 ↪columns. The columns parameter specifies the names of the columns in the␣
 ↪DataFrame.

meta=pd.DataFrame.
 ↪from_dict(xwalk,orient='index',columns=['wine','body_score','body_class'])
meta.index.name='ClassID'

feature_cols = [
    c for c in ann.columns
    if c not in ('SampleID', 'ClassID')
    and not c.strip().replace('.', '', 1).isdigit()   # removes unnamed BinBase␣
 ↪feature ( . . "226091.0")
] #it takes the metabolites columns only, 109 metabolites
X_wine = ann.groupby('ClassID')[feature_cols].mean()

data = meta.join(X_wine).reset_index()
data.shape

X=data.set_index('ClassID')[feature_cols]  #17 wines and 109 metabolites
check_zeros(X)
X_log=log_transform(X)
X_scaled=pareto_scale(X_log)
```

```
Total number of zeros in the dataset: 0
Total number of NaN values in the dataset: 0
```

```
Percentage of zeros in the dataset: 0.00%
Percentage of NaN values in the dataset: 0.00%
Number of columns with zeros: 0
head of the metabolites with the most zeros:
Series([], dtype: int64)
columns with at least one zero: 0/108
```

[4]:
```
data.shape
```

[4]:
```
(17, 112)
```

[5]:
```
X_scaled.head()
```

[5]:

ClassID	1,2-anhydro-myo-inositol	1-hexadecanol	2-hydroxyglutaric acid	\
421745	-0.170186	-0.229492	-0.181644	
421770	-0.030809	-0.100690	-0.127108	
421795	-0.376462	-0.648447	-0.009777	
421820	-0.354430	-0.066543	-0.145879	
421845	-0.336224	-0.609054	-0.469919	

ClassID	2-isopropylmalic acid	2-ketoisocaproic acid	\
421745	-0.120954	-0.358376	
421770	-0.055168	-0.300022	
421795	-0.320873	-0.571515	
421820	0.243210	-0.301254	
421845	-0.351755	-0.166290	

ClassID	3,6-anhydrogalactose	3-hydroxy-3-methylglutaric acid	\
421745	-0.294593	-0.579214	
421770	-0.542449	-0.094708	
421795	-0.005761	-0.482725	
421820	-1.084538	-0.267075	
421845	-0.522046	-0.280802	

ClassID	4-hydroxyproline	alanine	alpha ketoglutaric acid	…	\
421745	0.368172	0.155319	-0.078202	…	
421770	0.616162	0.410475	-0.256473	…	
421795	0.391114	-0.502665	0.025824	…	
421820	0.701855	0.736470	0.126066	…	
421845	0.030803	-0.130612	-0.374954	…	

ClassID	trehalose	tryptophan	tyrosine	uracil	urea	uridine	\
421745	-0.820861	-0.643022	0.003073	-0.401726	-0.248507	0.027379	

```
421770    -0.873084    -1.402195   0.384504 -0.506043  0.246881  0.914218
421795     0.426313     0.174806  -0.315254 -0.387872 -0.454005 -0.316768
421820    -1.148155     0.181480   0.596409  0.128222  0.147962  0.203450
421845    -0.733970     0.524797   0.443453  0.076441 -0.544899  0.480931

          valine   xanthine   xylitol    xylose
ClassID
421745   0.155285   0.163349 -0.391314  0.456157
421770   0.629854   0.350728 -0.132808  0.443070
421795  -0.329538  -0.007820 -0.628778  0.114289
421820   0.256917  -0.524722 -0.060760 -0.565442
421845   0.538127  -0.322962 -0.301289  0.385575

[5 rows x 108 columns]
```

[6]:
```
meta
```

[6]:
```
          wine  body_score body_class
ClassID
421745   CH01        2.57       high
421770   CH02        2.85       high
421795   CH03        3.21       high
421820   CH04        2.55       high
421845   CH05        2.91       high
421870   CH06        2.46     medium
421545   PG01        2.13        low
421570   PG02        2.24        low
421595    R01        2.44     medium
421620    R02        2.63       high
421645   SB01        2.16        low
421670   SB02        1.97        low
421695   SB03        2.33     medium
421720   SB04        2.12        low
421495    V01        2.42     medium
421520    V02        2.32     medium
421895   WW01        2.22        low
```

[ ]:
```

```

WE will do PLS because we have a higher number of deegrees of freedom and we need to find the latent components ,we also need to do LOO-CV and not the regular 5-fold because we have limited samples

[7]:
```
from sklearn.cross_decomposition import PLSRegression
from sklearn.model_selection import LeaveOneOut, cross_val_score
from sklearn.metrics import r2_score
#REGRESSION :THE OUTPUT FOR EACH WINE IS NUMBER
```

```
def loo_cv_q2(X:pd.DataFrame,y:pd.Series,n_components:int)->tuple[float,np.
 ↪ndarray]:

    loo=LeaveOneOut()
    y_pred=np.zeros(len(y))#we need it to save the 17 predictions for each␣
 ↪wine, we will use it to calculate the q2 score
    for train_index,test_index in loo.split(X):#loo.split is a generator that␣
 ↪yields the train and test indices for each iteration of the leave-one-out␣
 ↪cross-validation
        X_train,X_test=X.iloc[train_index],X.iloc[test_index]
        y_train,y_test=y.iloc[train_index],y.iloc[test_index]
        pls=PLSRegression(n_components=n_components)
        pls.fit(X_train,y_train)
        y_pred[test_index]=pls.predict(X_test).ravel() #ravel() flattens the␣
 ↪array to 1D


    q2=r2_score(y,y_pred)#it needs the prediction and the real values to␣
 ↪calculate the q2 score
    return q2,y_pred

def select_n_components(X: pd.DataFrame, y: pd.Series, max_components: int = 6)␣
 ↪-> pd.DataFrame:
    #it tries different components from 1 to max_components and returns a␣
 ↪dataframe with the q2 score for each number of components
    results = []
    for n in range(1, max_components + 1):
        q2, _ = loo_cv_q2(X, y, n)
        results.append((n, q2))
    return pd.DataFrame(results, columns=["n_components", "Q2"])

y=data.set_index('ClassID')['body_score']
scores_table=select_n_components(X_scaled,y,max_components=6)
print(scores_table)
```

```
   n_components        Q2
0             1  0.390782
1             2  0.277695
2             3  0.238403
3             4  0.177142
4             5  0.038023
5             6 -0.015506
```

[ ]:

[8]:
```
final_pls=PLSRegression(n_components=1) #we pick one because the first was the␣
 ↪best
final_pls.fit(X_scaled,y)
```

[8]:
```
PLSRegression(n_components=1)
```

[9]:
```
y
```

[9]:
```
ClassID
421745    2.57
421770    2.85
421795    3.21
421820    2.55
421845    2.91
421870    2.46
421545    2.13
421570    2.24
421595    2.44
421620    2.63
421645    2.16
421670    1.97
421695    2.33
421720    2.12
421495    2.42
421520    2.32
421895    2.22
Name: body_score, dtype: float64
```

[10]:
```
#R^2 Y calibration (in-sample ,always optimistic)

y_pred_calibration=final_pls.predict(X_scaled).ravel() #ravel() flattens the␣
 ↪array to 1D

y_pred_calibration

r2y_calibration = r2_score(y, y_pred_calibration)


#it will gives a scalar (the best component) and the predictions for each wine,␣
 ↪we will use it to calculate the q2 score
best_q2, y_pred_loo = loo_cv_q2(X_scaled, y, n_components=1)

print(f"R²Y (calibration, in-sample): {r2y_calibration:.3f}")
print(f"Q²  (LOO-CV, out-of-sample):  {best_q2:.3f}")
print(f"Overfitting gap (R²Y - Q²):   {r2y_calibration - best_q2:.3f}")
```

```
R²Y (calibration, in-sample): 0.681
Q²  (LOO-CV, out-of-sample):  0.391
```

```
Overfitting gap (R²Y - Q²):   0.291
```

[11]:
```
#Permutation test for Q² significance
def permutation_test_q2(X:pd.DataFrame, y:pd.
 ↪Series,n_components,n_permutations=1000,random_state=42):
    rng=np.random.default_rng(random_state)
    real_q2,_=loo_cv_q2(X,y,n_components)

#null distributionn :the metabolites have nothing to do with wine body. -> by␣
 ↪shuffling the y values we break the relationship between X and y, and we can␣
 ↪see how often we get a q2 as good as the real one by chance
    null_q2s=[]#we save the null q2s for each permutation
    for i in range(n_permutations):
        y_shuffled=y.copy()
        y_shuffled.values[:]=rng.permutation(y_shuffled.values) #shuffle the␣
 ↪values of y
        perm_q2,_=loo_cv_q2(X,y_shuffled,n_components)
        null_q2s.append(perm_q2)

    null_q2s=np.array(null_q2s)
    p_value=(1+(null_q2s>=real_q2).sum())/(1+n_permutations) #the p-value is␣
 ↪the proportion of null q2s that are greater than or equal to the real q2



    return real_q2,null_q2s,p_value

real_q2, null_q2s, p_value = permutation_test_q2(
    X_scaled, y, n_components=1, n_permutations=1000
)

print(f"Real Q² (LOO-CV):        {real_q2:.3f}")
print(f"Null mean ± std:         {null_q2s.mean():.3f} ± {null_q2s.std():.3f}")
print(f"Permutation p-value:     {p_value:.4f}")
```

```
Real Q² (LOO-CV):        0.391
Null mean ± std:         -0.536 ± 0.465
Permutation p-value:     0.0030
```

[12]:
```
print(f"Real Q² is {(real_q2 - null_q2s.mean()) / null_q2s.std():.2f} SD above␣
 ↪the null mean")
```

```
Real Q² is 1.99 SD above the null mean
```

we can predicth the actual score with p=0.003 and real predictive power

Q^2=0.30,n_components=1 was selected on the full dataset before permutation testing; the null distribution does not re-select n per shuffle. Given the observed effect (real Q^2 ~2 SD above null mean, p=0.003), this conservative bias does not threaten the conclusion.

We have a lower overfitting gap and a lower best q^2 because of missing many samples

PLSA-DA binary low vs high+medium it needs 0 or 1

[13]:
```
y_pred_calibration
```

[13]:
```
array([2.77779205, 2.80201075, 2.73694833, 2.75995266, 2.69045778,
       2.6550211 , 2.29209686, 2.2453794 , 2.52181702, 2.69165986,
       2.14321822, 2.16176644, 2.18157825, 2.11799409, 2.24481597,
       2.33670138, 2.17078985])
```

[14]:
```
from sklearn.cross_decomposition import PLSRegression
from sklearn.model_selection import LeaveOneOut
from sklearn.metrics import accuracy_score, balanced_accuracy_score

#we will categorize the wines based on their body score, into low ->0 and␣
 ↪medium+high->1
y_da=(meta['body_class']!='low').astype(int) #it makes a boolean and asks its␣
 ↪not low (true or false) ,as type int it will be 1 for true and 0 for false,␣
 ↪so we have a binary classification problem

y_da=y_da.loc[X_scaled.index] #we need to make sure that the index of y_da is␣
 ↪the same as X_scaled, so we will use the index of X_scaled to filter y_da


print("K         :")
y_da.value_counts() #it will show us how many wines are in each class, we have␣
 ↪7 low and 10 medium+high
```

```
K         :
```

[14]:
```
body_class
1    11
0     6
Name: count, dtype: int64
```

[15]:
```
majority_class_count=y_da.value_counts().max() #it will show us how many wines␣
 ↪are in the majority class, we have 10 medium+high
total=len(y_da) #total number of wines

majority_baseline=majority_class_count/total #it will show us the accuracy of␣
 ↪the majority class baseline
print(f"Majority classifier baseline accuracy: {majority_baseline:.3f}")
```

```
Majority classifier baseline accuracy: 0.647
```

Majority baseline (0.647): if we always say '1' without looking at any metabolite, we get 64.7% accuracy for free. PLS-DA must overcome this to be said to learn something.

[16]:
```
#LOO-CV for selecting n_components
#CLASSIFICATION :THE OUTPUT FOR EACH WINE IS BINARY (0 OR 1)
def loo_cv_da(X:pd.DataFrame,y:pd.Series,n_components:int):
    loo=LeaveOneOut()
    y_pred_class=np.zeros(len(y),dtype=int) #we need it to save the 17␣
 ↪predictions for each wine, we will use it to calculate the accuracy score

    for train_idx,test_idx in loo.split(X):
        X_train,X_test=X.iloc[train_idx],X.iloc[test_idx]
        y_train,y_test=y.iloc[train_idx],y.iloc[test_idx]

        pls=PLSRegression(n_components=n_components)
        pls.fit(X_train,y_train)

        y_hat=pls.predict(X_test).ravel()[0] #ravel() flattens the array to 1D,␣
 ↪we need the first element because it returns an array with one element
        y_pred_class[test_idx]=1 if y_hat>=0.5 else 0 #if the prediction is␣
 ↪greater than or equal to 0.5 we classify it as 1 (medium+high) else 0 (low)

    acc=accuracy_score(y,y_pred_class) #it needs the prediction and the real␣
 ↪values to calculate the accuracy score
    bal_acc=balanced_accuracy_score(y,y_pred_class) #it needs the prediction␣
 ↪and the real values to calculate the balanced accuracy score
    return acc,bal_acc,y_pred_class

print("\nComponent selection:")
for n in range(1, 5):
    acc, bal_acc, _ = loo_cv_da(X_scaled, y_da, n)
    print(f"  n_components={n}  acc={acc:.3f}  balanced_acc={bal_acc:.3f}")
```

```
Component selection:
  n_components=1  acc=0.882  balanced_acc=0.909
  n_components=2  acc=0.882  balanced_acc=0.871
  n_components=3  acc=0.941  balanced_acc=0.955
  n_components=4  acc=0.824  balanced_acc=0.826
```

[17]:
```
best_n_da = 3
final_acc, final_bal_acc, y_pred_final = loo_cv_da(X_scaled, y_da,␣
 ↪n_components=best_n_da)

print(f"E      n_components: {best_n_da}")
print(f"LOO Accuracy:          {final_acc:.3f}")
```

```
print(f"LOO Balanced Accuracy: {final_bal_acc:.3f}")
print(f"Majority baseline:     {y_da.value_counts().max()/len(y_da):.3f}")


#balanced accuracy is the average of recall obtained on each class. It is used␣
 ↪when the classes are imbalanced, as it gives equal weight to both classes.␣
 ↪In our case, we have 7 low and 10 medium+high, so the balanced accuracy is␣
 ↪more informative than the regular accuracy.
```

```
E      n_components: 3
LOO Accuracy:          0.941
LOO Balanced Accuracy: 0.955
Majority baseline:     0.647
```

[18]:
```
final_pls_da = PLSRegression(n_components=3)
final_pls_da.fit(X_scaled, y_da)
```

[18]:
```
PLSRegression(n_components=3)
```

[19]:
```
y
```

[19]:
```
ClassID
421745    2.57
421770    2.85
421795    3.21
421820    2.55
421845    2.91
421870    2.46
421545    2.13
421570    2.24
421595    2.44
421620    2.63
421645    2.16
421670    1.97
421695    2.33
421720    2.12
421495    2.42
421520    2.32
421895    2.22
Name: body_score, dtype: float64
```

[20]:
```
#the accuracies are very high so first we will do a permutation test to see if␣
 ↪the model is significant, we will permute the labels 1000 times and see how␣
 ↪many times we get an accuracy higher than the original accuracy, if it is␣
 ↪less than 5% then we can say that the model is significant
#we will not use the scipy permutation test because it expects a simple␣
 ↪statistic function that does 17 fits and splits and also does permutation on␣
 ↪x and not at y
```

```
#permutation test asks 'How well can this pipeline, with these X's and this␣
 ↪n=17, 'find' patterns that don't exist?'
def permutation_test_da(X:pd.DataFrame,y:pd.Series,n_components:
 ↪int=1000,n_permutations:int=1000,random_state:int=42):

    rng=np.random.default_rng(random_state) #it gives us a random number␣
 ↪generator that we can use to permute the labels, we will use it to shuffle␣
 ↪the labels 1000 times and see how many times we get an accuracy higher than␣
 ↪the original accuracy

    _,real_bal_acc,_=loo_cv_da(X,y,n_components) #it gives us the real balanced␣
 ↪accuracy of the model,loo_cv_da is a function that does leave-one-out␣
 ↪cross-validation and returns the accuracy and balanced accuracy of the␣
 ↪model, we will use it to get the real balanced accuracy of the model

    #null distribution
    null_bal_accs=[] #it will save the balanced accuracies of the permuted␣
 ↪labels, we will use it to create a null distribution of balanced accuracies,␣
 ↪we will use it to see how many times we get an accuracy higher than the␣
 ↪original accuracy
    for i in range(n_permutations):
        y_shuffled=y.copy()
        y_shuffled.values[:]=rng.permutation(y_shuffled.values) #it shuffles␣
 ↪the labels, we will use it to create a null distribution of balanced␣
 ↪accuracies, we will use it to see how many times we get an accuracy higher␣
 ↪than the original accuracy
        _, perm_bal_acc, _ = loo_cv_da(X, y_shuffled, n_components)
        null_bal_accs.append(perm_bal_acc)

    #this is a bollean array that tells us how many times we get an accuracy␣
 ↪higher than the original accuracy, we will use it to calculate the p-value
    null_bal_accs = np.array(null_bal_accs)
    p_value=(null_bal_accs >= real_bal_acc).mean()#it gives us the p-value of␣
 ↪the permutation test, we will use it to see if the model is significant, if␣
 ↪it is less than 0.05 then we can say that the model is significa



    print(f"Real balanced accuracy:   {real_bal_acc:.3f}")
    print(f"Null mean ± std:          {null_bal_accs.mean():.3f} ±␣
 ↪{null_bal_accs.std():.3f}")
    print(f"Permutation p-value:      {p_value:.4f}")

    return real_bal_acc, null_bal_accs, p_value

real_bal_acc, null_dist, p_val = permutation_test_da(
```

```
    X_scaled, y_da, n_components=3, n_permutations=1000
)
```

```
Real balanced accuracy:   0.955
Null mean ± std:          0.470 ± 0.151
Permutation p-value:      0.0010
```

the permutation p-value:if the metabolites had nothing to do with wine body, getting a score this high would happen roughly once in a thousand tries. It happened on our real data. The most reasonable conclusion is that the metabolites do carry real information about wine body.

#balanced accuracy is the average of recall obtained on each class. It is used when the classes are imbalanced, as it gives equal weight to both classes. #q^2 answers how close was my predicted number to the real (for regression) and balanced accuracy answers how well did my model classify the wines into low and medium+high (for classification)

#balanced accuracy(DA) has a floor #recap ,component is one axis through metabolite-space that best tracks body score.

#we can compare them directly thats why we used how many Standart Deviation (SD) above the null mean, because the null distribution is different for each model, so we need to standardize the results to compare them directly. #next we need to find VIP #with n_components=1 there is only one weight vector and it explains 100% of whatever y_variance the model captures ,every metabolite has a weight in that vector, the weight is the contribution of that metabolite to the model, the higher the weight the more important the metabolite is for the model, we will use VIP to find the most important metabolites for the model

#because we want to answer which metabolites track the continuous body score we will use the regression model with n_components=1 to find the VIP scores, we will use the weights of the model to calculate the VIP scores

[21]:
```
#VIP scores for the regression model (final_pls, n_components=1)
def compute_vip(pls_model, X: pd.DataFrame, y: pd.Series) -> pd.Series:
    T = pls_model.x_scores_      # (17, n_components) - how each wine projects␣
 ↪onto each component
    W = pls_model.x_weights_     # (108, n_components) - how each metabolite␣
 ↪loads onto each component
    Q = pls_model.y_loadings_    # (1, n_components) - how much each component␣
 ↪explains y

    p, A = W.shape                # p=108 metabolites, A=number of components

    SSY = np.sum(T**2, axis=0) * (Q.ravel()**2)   #variance in y explained by␣
 ↪each component
    total_SSY = SSY.sum()

    vip = np.zeros(p)
    for j in range(p):
        weight = 0.0
```

```
        for a in range(A):
            w_ja_norm = W[j, a] / np.linalg.norm(W[:, a])   #normalize weight␣
 ↪vector to unit length
            weight += (w_ja_norm ** 2) * SSY[a]
        vip[j] = np.sqrt(p * weight / total_SSY)

    return pd.Series(vip, index=X.columns, name='VIP').
 ↪sort_values(ascending=False)


vip_scores = compute_vip(final_pls, X_scaled, y)

print(vip_scores.head(15))
print(f"\nMetabolites with VIP > 1: {(vip_scores > 1).sum()} out of␣
 ↪{len(vip_scores)}")
```

```
2-ketoisocaproic acid               2.008796
palmitic acid                       1.937752
1-hexadecanol                       1.863199
stearic acid                        1.848420
octadecanol                         1.837456
tartaric acid                       1.744982
arachidic acid                      1.713181
ribose                              1.677779
inositol myo-                       1.617277
4-hydroxyproline                    1.610280
3-hydroxy-3-methylglutaric acid     1.589415
sophorose                           1.573052
isocitric acid                      1.562852
glycerol                            1.552783
heptadecanoic acid                  1.552246
Name: VIP, dtype: float64

Metabolites with VIP > 1: 45 out of 108
```

fatty acids dominate and they influence mouthcoating and texture perception

[22]:
```
#PLS component signs are arbitrary lets anchor to y before reading directions
scores_c1 = final_pls.x_scores_[:, 0]
sign = np.sign(np.corrcoef(scores_c1, y)[0, 1])   #+1 if component tracks␣
 ↪body_score, -1 if inverted

print(f"Correlation between component 1 scores and body_score: {np.
 ↪corrcoef(scores_c1, y)[0,1]:.3f}")
print(f"Sign correction applied: {sign:+.0f}")

loadings = pd.Series(final_pls.x_weights_[:, 0] * sign, index=X_scaled.columns,␣
 ↪name='weight')
```

```
vip_table = pd.DataFrame({
    'VIP': vip_scores,
    'weight': loadings.loc[vip_scores.index],
    'direction': np.where(loadings.loc[vip_scores.index] > 0, 'higher body','lower body')
})

print(vip_table.head(15))

vip_table.to_csv('../results/tables/phase5_vip_scores.csv')
```

```
Correlation between component 1 scores and body_score: -0.826
Sign correction applied: -1
                                      VIP    weight    direction
2-ketoisocaproic acid            2.008796 -0.193297   lower body
palmitic acid                    1.937752 -0.186460   lower body
1-hexadecanol                    1.863199 -0.179286   lower body
stearic acid                     1.848420 -0.177864   lower body
octadecanol                      1.837456 -0.176809   lower body
tartaric acid                    1.744982 -0.167911   lower body
arachidic acid                   1.713181 -0.164851   lower body
ribose                           1.677779 -0.161444   lower body
inositol myo-                    1.617277 -0.155623   lower body
4-hydroxyproline                 1.610280  0.154949  higher body
3-hydroxy-3-methylglutaric acid  1.589415 -0.152942   lower body
sophorose                        1.573052 -0.151367   lower body
isocitric acid                   1.562852 -0.150386   lower body
glycerol                         1.552783 -0.149417   lower body
heptadecanoic acid               1.552246 -0.149365   lower body
```

The fatty acid story makes sense in the direction the data actually shows. The paper explains the mechanism-> palmitic and stearic acids are among the most abundant fatty acids in grapes, and their concentration declines throughout fermentation. So high fatty acid levels are a signature of less complete fermentation and fuller-bodied wines tend to be the more thoroughly fermented, more extracted ones. The fatty acids aren't causing thin body but they're a marker of the winemaking trajectory that produces it.

[23]:
```
import mlflow, hashlib
from mlflow.tracking import MlflowClient
client = MlflowClient()
r = client.get_run("a2158348a5ab45259e62850a855ef6b6")
print(r.data.metrics["q2_loo"])
print(r.data.params["source_sha256"])
MLFLOW_URI = "sqlite:////home/nasia/wine-innovation-engine/notebooks/mlflow.db"
mlflow.set_tracking_uri(MLFLOW_URI)
mlflow.set_experiment("phase5_metabolomics")
```

```
SOURCE_XLS = "/home/nasia/wine-innovation-engine/data/processed/summary_421340.
 ↪xls"

def sha256_file(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(8192), b""):
            h.update(chunk)
    return h.hexdigest()

client = MlflowClient()
run = client.create_run(mlflow.get_experiment_by_name("phase5_metabolomics").
 ↪experiment_id)
rid = run.info.run_id

#params
params = {
    "dataset": "ST000006 (Skogerson 2009, GC-TOF-MS)",
    "source_sha256": sha256_file(SOURCE_XLS),
    "n_samples": 17,
    "n_features": X_scaled.shape[1],
    "aggregation": "6 injections -> 1 biological unit (mean)",
    "preprocessing": "BinBase-normalized -> log10(x+1) -> Pareto scaling",
    "scaling_scope": "full dataset (documented simplification)",
    "cv": "LeaveOneOut",
    "n_components_regression": 1,
    "n_components_da": 3,
    "n_permutations": 1000,
    "random_state": 42,
}
for k, v in params.items():
    client.log_param(rid, k, v)

#metrics
metrics = {
    "r2y_calibration": r2y_calibration,
    "q2_loo": real_q2,
    "overfitting_gap": r2y_calibration - real_q2,
    "q2_perm_pvalue": p_value,
    "q2_null_mean": float(null_q2s.mean()),
    "q2_null_std": float(null_q2s.std()),
    "q2_sd_above_null": float((real_q2 - null_q2s.mean()) / null_q2s.std()),
    "da_accuracy": final_acc,
    "da_balanced_accuracy": final_bal_acc,
    "da_majority_baseline": float(y_da.value_counts().max() / len(y_da)),
    "n_vip_above_1": int((vip_scores > 1).sum()),
```

```
    "component1_corr_with_y": float(np.corrcoef(final_pls.x_scores_[:, 0],␣
 ↪y)[0, 1]),
}
for k, v in metrics.items():
    client.log_metric(rid, k, v)

#artifacts
vip_table.to_csv("/tmp/phase5_vip_scores.csv")
np.save("/tmp/phase5_null_q2.npy", null_q2s)
client.log_artifact(rid, "/tmp/phase5_vip_scores.csv")
client.log_artifact(rid, "/tmp/phase5_null_q2.npy")

client.set_terminated(rid)
print(f"Logged run: {rid}")
```

```
0.3907820232632142
3ff12ab813e6b3e7e596987a207fd5315a3573d8c944808427b986e9a782accd

Logged run: a56b5c5e23a049bdb41eeedd70abfd2a
```